

TECHNICAL ARCHITECTURE

PROJECT SENTINEL

Autonomous AI-Powered Payment Risk Intelligence & Protection Platform

Combining Machine Learning, Graph Intelligence, Agentic AI and PayPal Payment Infrastructure

PREPARED FOR

PayPal AI Hackathon 2026

PREPARED BY

Tanisha Dey

DATE

October 2026

Disclaimer: Sentinel is a hackathon prototype. It runs on PayPal Sandbox with synthetic data only and is not an official, deployed or endorsed PayPal fraud-prevention product. It does not access PayPal's internal risk systems. Model results describe synthetic data and are not evidence of real-world performance.

About this document

Architecture of the implemented system: components, transaction lifecycle, component interaction, data model, API flow, ML inference, agent tool calling, graph intelligence, payment state machine, security boundaries and deployment. Generated from docs/SENTINEL_TECHNICAL_ARCHITECTURE.md (Mermaid diagrams are drawn natively here).

Item	Detail
Project	PROJECT SENTINEL: Autonomous AI-Powered Payment Risk Intelligence & Protection Platform
Prepared for	PayPal AI Hackathon 2026
Prepared by	Tanisha Dey
Source of truth	The repository (backend/, frontend/, docs/). Every metric and test result was taken from the running system or its test suite.
Repository	github.com/Tanisha-dey18/hackathon
Generated	2026-11-01

> **Disclaimer.** Sentinel is a hackathon prototype using PayPal Sandbox and synthetic data. It is not an officially deployed PayPal fraud prevention product and has no access to PayPal's internal systems. Features that are designed but not implemented are labelled **Future Scope**, and integrations that are implemented but were not executed live are labelled as such.

Contents

SENTINEL Technical Architecture 4

1. High-level architecture 4
2. Transaction lifecycle 4
3. Component interaction 5
4. Database ER diagram 5
5. API communication flow 6
6. ML inference flow 6
7. AI agent tool-calling flow 7
8. Graph intelligence 8
9. PayPal payment flow and state 9
10. Security boundaries and data flow 9
11. Deployment 9

SENTINEL

SENTINEL Technical Architecture

This document describes the system as implemented in this repository (`backend/app`, `frontend/src`). Every diagram matches the code. Items that are configured but not executed, or not implemented, are marked as such.

1. High-level architecture

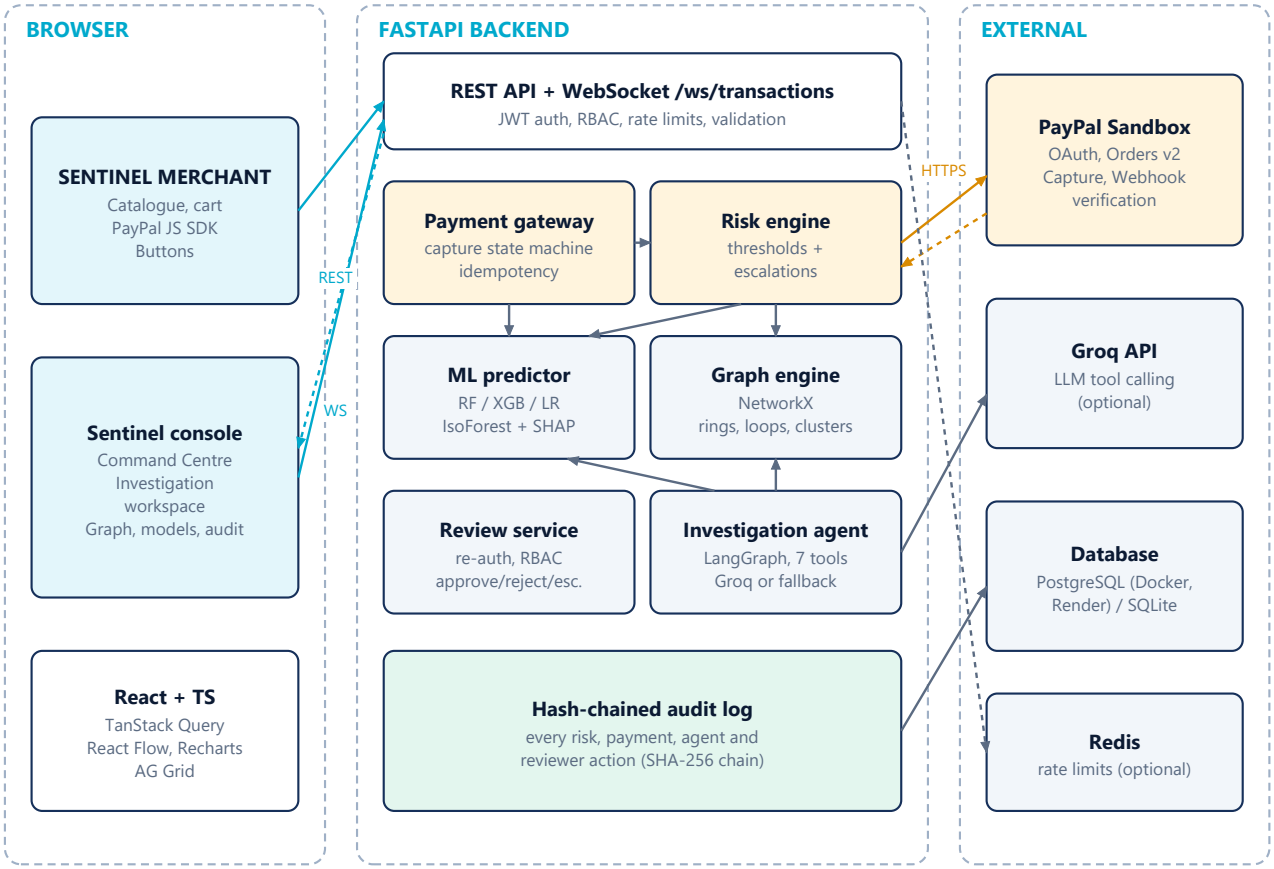


Figure 1: High-level system architecture

The browser runs one React application with two areas: the storefront for shoppers and the console for staff. The FastAPI backend holds all business logic and is the only component that talks to PayPal and Groq. State lives in a relational database: PostgreSQL in Docker/Render, SQLite for local development. Redis is optional and only used for shared rate-limit counters.

2. Transaction lifecycle

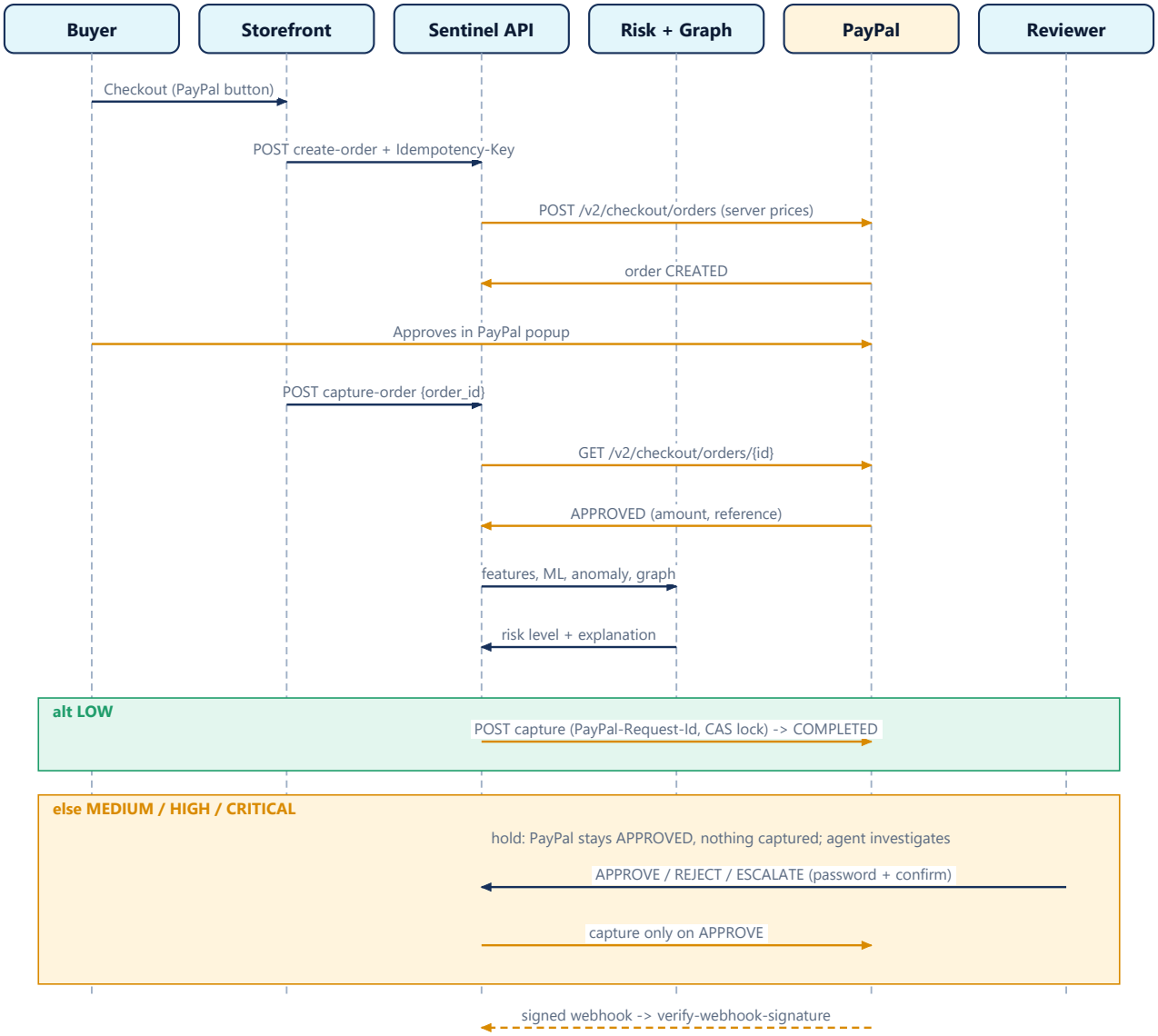


Figure 2: Payment processing sequence (Orders v2 with Sentinel risk gating)

3. Component interaction

Both capture paths go through the same `execute_capture()`. That function re-checks authorisation itself (a LOW assessment for `AUTO_POLICY`, or a stored APPROVE decision for `REVIEWER`) before it takes the lock and calls PayPal.

4. Database ER diagram

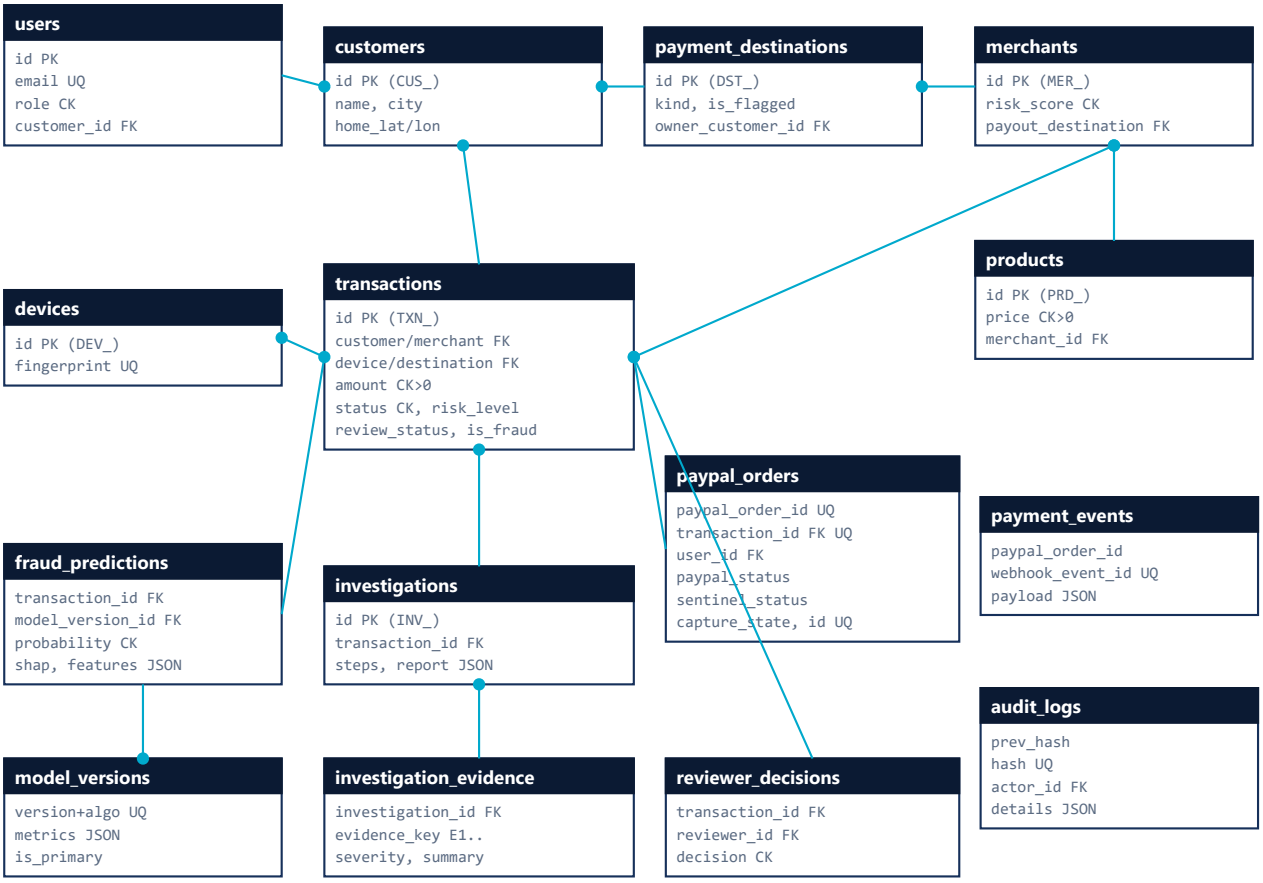


Figure 3: Database relationships (main tables and foreign keys)

Key constraints: `transactions.amount > 0`, allowed status lists, unique `paypal_orders.paypal_order_id` / `transaction_id` / `create_idempotency_key` / `capture_id`, unique `payment_events.webhook_event_id`, unique `audit_logs.hash`, and composite indexes on customer+time and device+time. `payment_events` and `system_settings` have no foreign keys (events are keyed by PayPal order ID).

5. API communication flow

Requests carry `Authorization: Bearer <JWT>`. Validation errors return 422 with a structured list. PayPal errors return 502 with PayPal's `debug_id`. A missing PayPal configuration returns 503. Background work (investigations, simulator, retraining) runs in threads and pushes WebSocket events with `asyncio.run_coroutine_threadsafe`.

6. ML inference flow

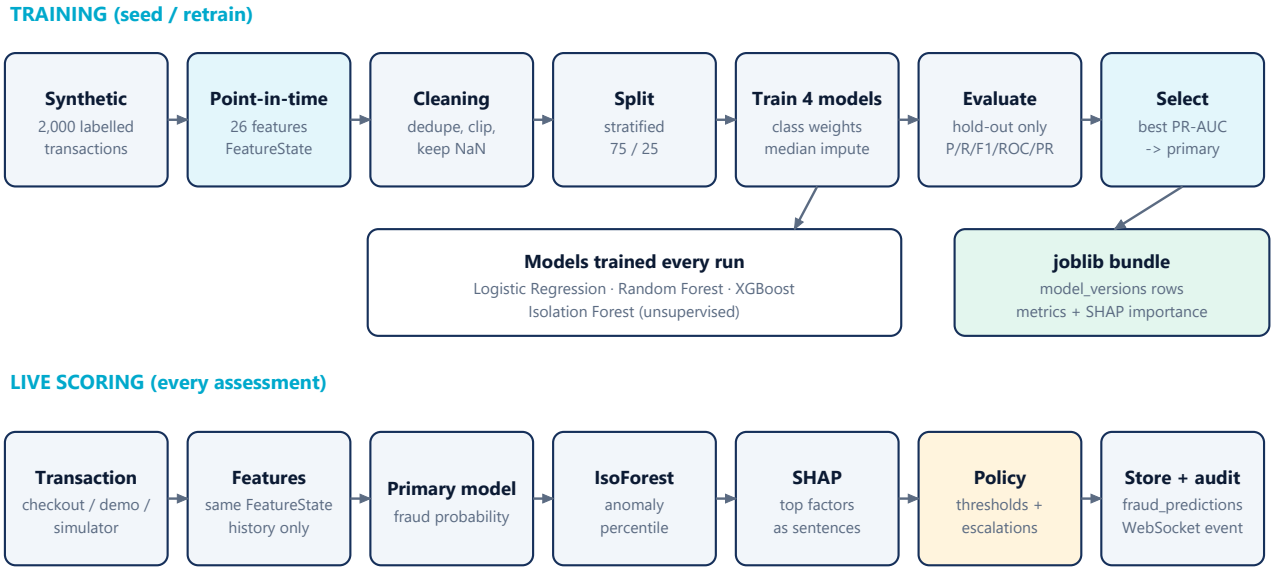


Figure 4: Machine-learning fraud detection pipeline

Training replays history chronologically through the same `FeatureState`, uses a stratified 75/25 split and class weighting, trains four models and selects the primary by hold-out PR-AUC (v1: Random Forest, PR-AUC 0.914 on synthetic data).

7. AI agent tool-calling flow

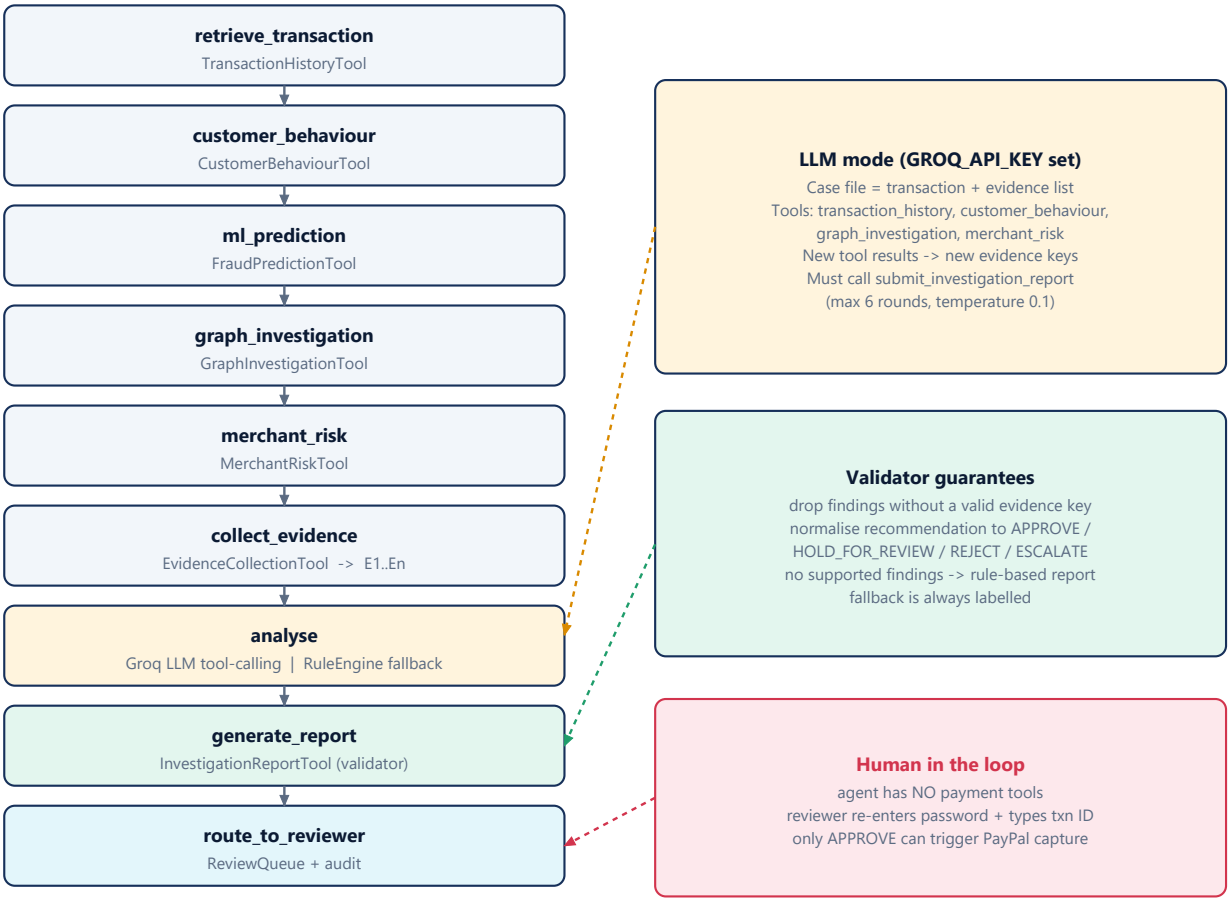


Figure 5: LangGraph investigation agent workflow

The validator drops findings that cite no valid evidence key, normalises the recommendation, and falls back to the rule-based report if no supported findings remain. The agent has no payment tools. **The Groq path is implemented but has not been executed** (no API key was available); all recorded investigations used the labelled fallback.

8. Graph intelligence

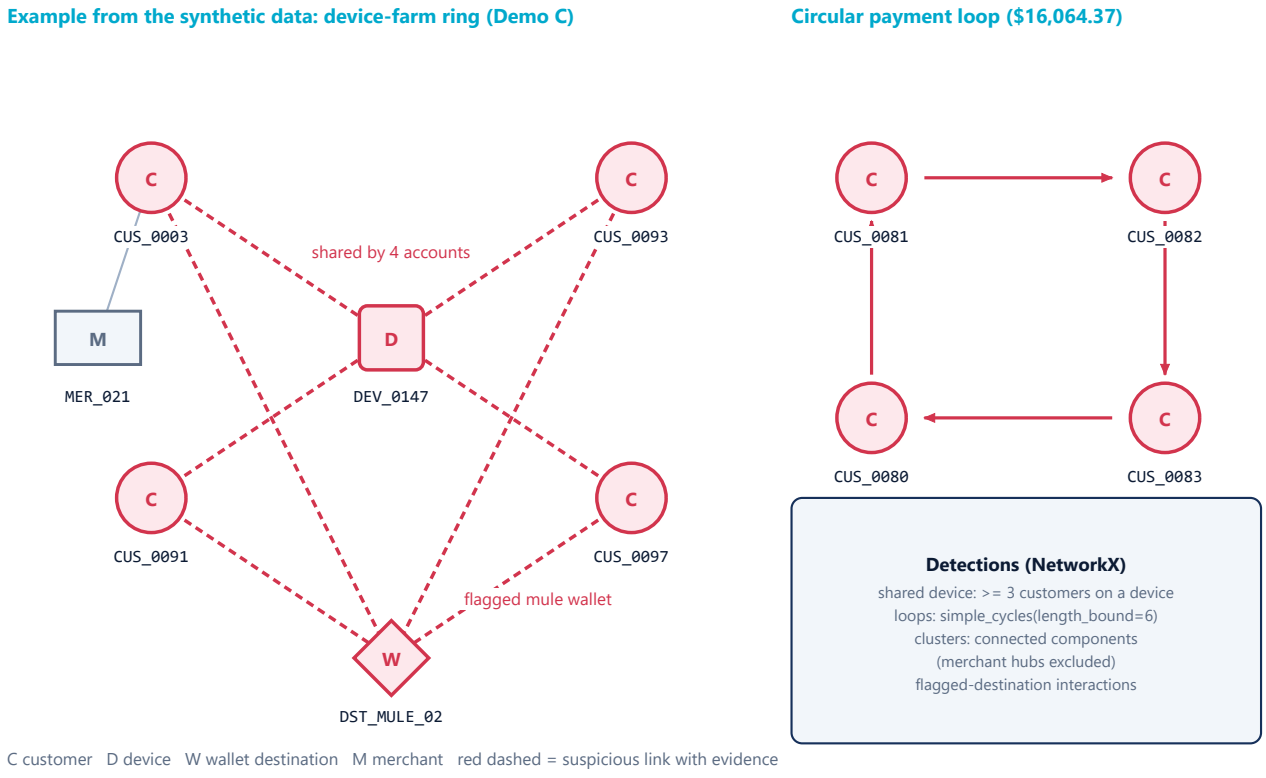


Figure 6: Graph intelligence: suspicious structures found in the synthetic data

Community detection is **not implemented**; connected components are used instead.

9. PayPal payment flow and state

This diagram shows `sentinel_status`. `paypal_status` / `capture_status` are separate fields written only from PayPal responses and verified webhooks.

10. Security boundaries and data flow

Boundary	Control
Browser to API	JWT, RBAC, Pydantic validation, rate limits, CORS allow-list, security headers
Client-supplied prices	Ignored; recalculated server-side, then re-verified against PayPal
Order ownership	Every order endpoint checks `user_id` (or staff role where allowed)
PayPal secret	Server environment only; browser receives only the client ID
Webhooks	Accepted only after PayPal's verify-webhook-signature returns SUCCESS; event IDs de-duplicated
Irreversible actions	Only humans (re-authenticated) can approve held captures; the agent has no payment tools
Audit	Hash-chained rows; `/api/audit/verify` detects modification

11. Deployment

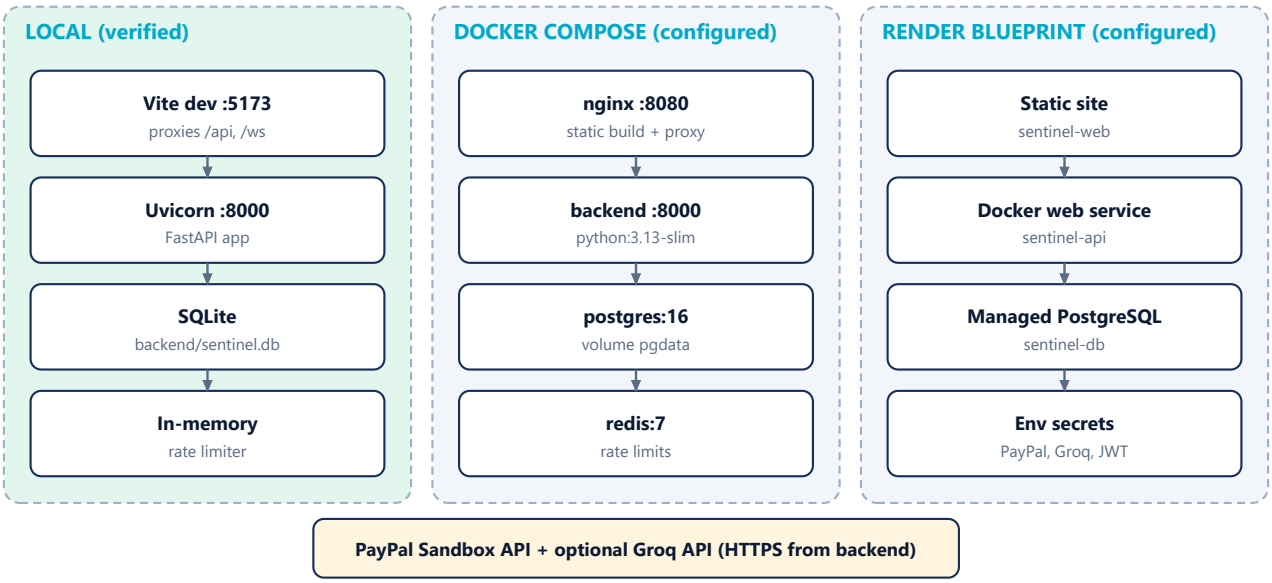


Figure 7: Deployment architecture

Mode	Status
Local: Uvicorn + Vite, SQLite, in-memory limits	Running and verified
Docker Compose: postgres:16, redis:7, backend, nginx frontend	Configured, not run on the build machine
Render Blueprint: managed PostgreSQL, Docker API, static site	Configured, not deployed